

3 отчёт по внешнему курсу

Выполнение работы

Цель работы

Научиться работе в ос Linux на базовом уровне.

Задание

Выполнить курс на stepic. 2 этап

Теоретическое введение

Курс выполнялся на платформе stepic.

Выполнение лабораторной работы

Выполнение 1 задания (рис. 1).

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

☒ Правильно, молодец!

- ☐ "Esc"
- ☐ "Ctrl", затем "x"
- ☐ ":", затем "q"
- ☒ ":", затем "q", затем "Enter"
- ☐ "Q"

Следующий шаг

Решить снова

Задание 1

Выполнение 2 задания (рис. 2).

При перемещении в vim "по словам" есть небольшая разница в том, используем мы маленькую (w, e, b) или большую (W, E, B) букву. Первые перемещают нас по "словам" (word), а вторые по "большим словам" (WORD). Посмотрите справку по этим перемещениям и разберитесь в чем заключается разница между word и WORD.

А для того, чтобы убедиться, что вы разобрались, отметьте ниже **все верные** утверждения про следующую строку:

```
Strange_ TEXT is_here. 2=2 YES!
```

Примечание: во всех утверждениях имеется ввиду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Подсказка: чтобы вызвать **vim-справку** по, например, перемещению `w`, нужно открыть vim и ввести команду `:help w`. Вы попадете в то место справки, где описано это перемещение, а так как все перемещения описаны рядом, то двигаясь по тексту вверх и вниз можно прочитать и про `e` и про `b` и, самое главное, про word и WORD. Кроме того, можно вызвать сразу справку по термину word при помощи `:help word`. Чтобы закрыть справку, нужно ввести команду `:q`.

Выберите все подходящие ответы из списка

 Отлично!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Нажимая только на W, нельзя переместить курсор на "."
- ☒ В этой строке 5 "больших слов" (WORD)
- ☐ В этой строке 12 "слов" (word)
- ☒ В этой строке 9 "слов" (word)
- ☒ Чтобы попасть в конец строки, нужно совершить меньше нажатий на W, чем на w
- ☒ После 10 нажатий на W курсор окажется там же, где бы он был после 10 нажатий на w

Следующий шаг

Решить снова

Задание 2

Выполнение 3 задания (рис. 3).

Предположим, что в текстовом файле записана одна единственная строка:

```
one two three four five
```

и вам нужно преобразовать её в строку

```
three four four four five
```

Какие(ой) из предложенных ниже наборов нажатий клавиш выполнят такое редактирование? В этих наборах нажатие на клавишу Esc обозначается как <Esc> (т.е. знаки "<" и ">" не несут отдельного смысла).

Примечание: во всех утверждениях имеется в виду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Выберите все подходящие ответы из списка

☒ Верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ x2wwywPp
- ☒ d2wwywPp
- ☒ ddithree four four five<Esc>
- ☒ d2wwifour four <Esc>
- ☐ d2dwywPp
- ☐ d2wwywpp

Следующий шаг

Решить снова

Задание 3

Выполнение 4 задания (рис. 4).

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. **включая** ввод ":" в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Напишите текст

☒ Верно. Так держать!

```
:%s/Windows/Linux
```

Следующий шаг

Решить снова

Задание 4

Выполнение 5 задания (рис. 5).

Мы совсем не рассказали вам про третий режим работы vim – режим **выделения (Visual)**. Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в vimtutor, который мы настоятельно рекомендуем вам для изучения vim!

Чтобы убедиться, что вы разобрались с этим режимом работы, отметьте, пожалуйста, **все верные** утверждения из списка ниже.

Подсказка: если вы не хотите проходить vimtutor целиком, то можете открыть его и поиском найти слово **"Visual"**. Вы попадете в задание, прохождение которого будет вполне достаточно, чтобы выполнить это задание.

Выберите все подходящие ответы из списка

✓ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Когда вы находитесь в режиме выделения, внизу редактора горит надпись – VISUAL – (или – ВИЗУАЛЬНЫЙ РЕЖИМ –)
- ☒ В режиме выделения можно использовать команды d (удалить) и u (скопировать)
- ☐ Режим выделения открывается при помощи команды :visual
- ☐ Чтобы выйти из режима выделения, нужно ввести :q
- ☒ Выйти из режима выделения можно, нажав клавишу Esc два раза
- ☒ В режиме выделения можно использовать команды перемещения (например, W, e, \$, и др.)

Следующий шаг

Решить снова

Задание 5

Выполнение 6 задания (рис. 6).

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попытаете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка

✓ Хорошая работа.

- ☐ Никакие команды появляться не будут
- ☒ Только из набора C
- ☐ Из наборов A и C
- ☐ Только из набора A
- ☐ Из наборов B и C

Следующий шаг

Решить снова

Задание 6

Выполнение 7 задания (рис. 7).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#), [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
touch file1.txt
cd /home/bi/Desktop/
```

Как будет выглядеть **абсолютный путь** до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один вариант из списка

✓ Правильно.

- ☐ `/home/bi/Desktop/file1.txt`
- ☐ `/home/bi/Documents/file1.txt`
- ☒ `/home/bi/file1.txt`
- ☐ Ничкак (файла `file1.txt` не будет существовать после завершения работы скрипта)

Следующий шаг

Решить снова

Задание 7

Выполнение 8 задания (рис. 8).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк **могут** быть именами переменных в `bash`? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Выберите все подходящие ответы из списка

✓ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `variable`
- ☐ `123variable`
- ☐ `var@iabile`
- ☒ `_variable`
- ☒ `__variable`
- ☒ `variable123`
- ☒ `VARiable`

Следующий шаг

Решить снова

Задание 8

Выполнение 9 задания (рис. 9).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [arguments.sh](#).

Напишите скрипт на bash, который принимает на вход два аргумента и выводит на экран строку следующего вида:

```
Arguments are: $1=первый_аргумент $2=второй_аргумент
```

Например, если ваш скрипт называется `./script.sh`, то при запуске его `./script.sh one two` на экране должно появиться:

```
Arguments are: $1=one $2=two
```

а при запуске `./script.sh three four` будет:

```
Arguments are: $1=three $2=four
```

Подсказка: в случае проблем с решением задачи, обратите внимание [на наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✓ Абсолютно точно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2 var1=$1
3 var2=$2
4
5 echo "Arguments are: \${1=$var1} \${2=$var2}"
6
7
8
```

Следующий шаг

Решить снова

Задание 9

Выполнение 10 задания (рис. 10).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [branching1.sh](#).

Предположим, вы пишете скрипт на `bash` и хотите использовать в нем конструкцию `if` в следующем фрагменте:

```
if [[ ... ]]
then
  echo "True"
fi
```

Вы можете вписать вместо `"..."` (внутри `[[ ]]` и **не забудьте про пробелы** после `[[` и перед `]]`!) любое из перечисленных ниже условий. Однако мы просим вас выбрать только те из них, при которых `echo` напечатает на экран `True` вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные.

Например, условие `0 -eq 0` **подходит**, т.к. ноль всегда равен нулю вне зависимости от аргументов и переменных внутри скрипта и на экран будет напечатано `True`. В то же время условие `$var1 -eq 0` **не подходит**, так как в переменной `var1` как может быть записан ноль (тогда будет напечатано `True`), так его может и не быть (тогда ничего напечатано не будет).

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Выберите все подходящие ответы из списка

☒ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `0 -ge 0`
- ☒ `-s $0`
- ☒ `-n $0`
- ☐ `0 -gt 0`
- ☒ `5 -ge 5`
- ☐ `-z ""`

Следующий шаг

Решить снова

Задание 10

Выполнение 11 задания (рис. 11).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [branching2.sh](#), [branching3.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
if [[ $var -gt 5 ]]
then
  echo "one"
elif [[ $var -lt 3 ]]
then
  echo "two"
elif [[ $var -eq 4 ]]
then
  echo "three"
else
  echo "four"
fi
```

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную `var=3`, а затем запустили еще раз, но уже с `var=5`.

Выберите один вариант из списка

☒ Верно.

- ☐ Сначала two, потом four
- ☐ Сначала four, потом one
- ☐ Сначала one, потом two
- ☒ Сначала four, потом four

Следующий шаг

Решить снова

Задание 11

Выполнение 12 задания (рис. 12).

Напишите скрипт на `bash`, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории. В зависимости от значения числа нужно вывести разные сообщения.

Соответствие входа и выхода должно быть таким:

```
0 --> No students
1 --> 1 student
2 --> 2 students
3 --> 3 students
4 --> 4 students
5 и больше --> A lot of students
```

Примечание а): выводить нужно только строку справа, т.е. "-" выводить не нужно.

Примечание б): в последней строке слово "lot" с маленькой буквы!

Примечание 2: в этой и всех последующих задачах на написание скриптов, если не указано явно, что нужно проверять вход (например, что он будет именно числом и именно от 0 до бесконечности), то этого делать не нужно!

Пример №1: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 1` на экране должно появиться:

```
1 student
```

Пример №2: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 5` на экране должно появиться:

```
A lot of students
```

Подсказка: в случае проблем с решением задачи, обратите внимание на наши [рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через `stdin` → `stdout`

✓ Хорошие новости, верно!

Теперь вам доступен [форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 if [[ $1 -eq 1 ]]; then
2   echo "1 student"
3 elif [[ $1 -gt 1 && $1 -le 4 ]]; then
4   echo "$1 students"
5 elif [[ $1 -ge 5 ]]; then
6   echo "A lot of students"
7 else
8   echo "No students"
9 fi
10
11
12
13
14
```

Следующий шаг

Решить снова

Задание 12

Выполнение 13 задания (рис. 13).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [loops1.sh](#), [loops2.sh](#).

Посмотрите на фрагмент `bash`-скрипта:

```
for str in a , b , c_d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово "start", а сколько раз слово "finish"?

Выберите один вариант из списка

✓ Правильно, молодец!

- ☒ 5 раз "start" и 4 раза "finish"
- ☐ 5 раз "start" и 5 раз "finish"
- ☐ 3 раза "start" и 2 раза "finish"
- ☐ 5 раз "start" и 2 раза "finish"

Следующий шаг

Решить снова

Задание 13

Выполнение 14 задания (рис. 14).

Напишите скрипт на Bash, который будет определять в какую возрастную группу попадает пользователь. При запуске скрипта должен вывестись сообщение "Enter your name" и ждать от пользователя ввода имени (используя `read`), чтобы прочитать его). Когда имя введено, то скрипт должен написать "Enter your age" и ждать ввода возраста (используя `read`). Когда возраст введён, скрипт пишет на экран "Ok, my, your group is group", где группа определяется на основе возраста по следующим правилам:

- меньше либо равно 16: 'child';
- от 17 до 24 (включительно): 'youth';
- старше 25: 'adult'.

После этого скрипт опять выводит сообщение "Enter your name" и всё начинается по-новому (бесконечный цикл). Если в какой-то момент работы скрипта будет введено пустое имя или возраст 0, то скрипт должен написать на экран "bye" и закончить свою работу (выйти из цикла).

Примеры корректной работы скрипта:

№1

```
./script.sh
Enter your name:
Enter your age:
16
Enter your group is child
Enter your name:
Elena Petровна
Enter your age:
0
bye
```

№2

```
./script.sh
Enter your name:
Elena Petровна
Enter your age:
25
Elena Petровна, your group is youth
Enter your name:
bye
```

Подсказка: в случае проблем с решением задания, обратитесь внимание на наши рекомендации по написанию скриптов.

Напишите программу. Тестируется через stdin → stdout

✓ **Правильно, молодцы!**

Теперь ваш скрипт **фактически решен!**, вы можете сравнить свое решение с другими или отправить совет.

Вы решили задачу **всего за 1 минуту 30 секунд**. Вы можете получить дополнительные **комментарии**, оставив на них вопрос, или сравнить свой решение с другими на **форуме решений**.

```
1 #!/usr/bin/bash
2 #
3
4 while [[ $x != 1 ]]
5 do
6
7 echo "enter your name:"
8 read name
9 if [[ -x $name ]]
10 then echo "bye"
11
12 break
13 fi
14 echo "enter your age:"
15 read age
16 if [[ $age -eq 0 ]]
17 then echo "bye"
18
19 break
20 elif [[ $age -le 16 ]]
21 then echo "$name, your group is child"
22 elif [[ ($age -le 24) && ($age -ge 17) ]]
23 then echo "$name, your group is youth"
24 else echo "$name, your group is adult"
25
26 fi
27 fi
```

Задание 14

Выполнение 15 задания (рис. 15).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [math1.sh](#), [math2.sh](#).

Какие(ая) из предложенных ниже инструкций увеличат значение переменной `a` на значение переменной `b`? Например, если в `a` было записано 10, в `b` было 5, то в `a` должно записаться 15.

Выберите **все подходящие** варианты!

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Подсказка: обратите особое внимание на кавычки и **пробелы**, они могут как принципиально изменить команду, так и ни на что не повлиять (в зависимости от команды и контекста)!

Выберите все подходящие ответы из списка

☒ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `let "a = a + b"`
- ☒ `let "a=${a+$b}"`
- ☒ `let "a+=b"`
- ☒ `let a=a+b`
- ☐ `a=${a+$b}`

Следующий шаг

Решить снова

Задание 15

Выполнение 16 задания (рис. 16).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [programs.sh](#).

Пусть вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
echo "`pwd`"
```

Что в этом случае выведет команда `echo` на экран?

Выберите один вариант из списка

☒ Хорошая работа.

- ☐ ``pwd``
- ☐ Код возврата команды `pwd` (0 в случае успешного выполнения и не 0 в случае ошибок)
- ☐ `/home/bi/Documents`
- ☐ `pwd`
- ☒ `/home/bi`

Следующий шаг

Решить снова

Задание 16

Выполнение 17 задания (рис. 17).

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции `if` при помощи `if `program options arguments`` (действия внутри `if` выполняются, если программа закончилась с кодом 0). Однако это **не всегда правда**! Если запуск внешней программы выводит что-то в `stdout`, то в проверку `if` поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой `bash`-скрипт с использованием, например, `if `pwd``.

Однако как быть, если хочется всё-таки запустить программу `program`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если ее код возврата равен 0? Выберите **все верные** утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, **используется именно косая кавычка** (`'`), а не обычная (`"`) или двойная (`"`).

Выберите все подходящие ответы из списка

✓ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `if `program > some_file.txt``
- ☐ `if [[ `program` -eq 0 ]]`
- ☐ Ничего сделать нельзя
- ☐ Сначала `var=`program``, затем `if [[ $var -eq 0 ]]`
- ☒ Сначала запустить `program`, затем `if [[ $? -eq 0 ]]`

Следующий шаг

Решить снова

Задание 17

Выполнение 18 задания (рис. 18).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [functions1.sh](#), [functions2.sh](#).

Посмотрите на функцию из `bash`-скрипта:

```
counter () # takes one argument
{
    local let "c1+=$1"
    let "c2+=${1}*2"
}
```

Впишите в форму ниже **строку**, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте **после десяти вызовов** функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое 🤔). В этом случае имеет смысл написать небольшой скрипт на `bash`, который проделает ровно то, что указано в задании и посимвольно сверить свой ответ с тем, что он выдаст на экран.

Напишите текст

✓ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

counters are and 110

Следующий шаг

Решить снова

Задание 18

Выполнение 19 задания (рис. 19).

Напишите скрипт на `bash`, который будет искать наибольший общий делитель (`НОД`, `greatest common divisor`, `GCD`) двух чисел. При запуске ваш скрипт не должен ничего писать на экран, а просто выводить результат работы через `printf` (для этого можно использовать `%d %d`) и указывать ему два параметра — `оп.`, пример в видеоброфайле). После ввода чисел скрипт считывает из `STDIN` и выводит на экран сообщение "`GCD` `из` `исходительных` значений", например для чисел 15 и 25 это будет "`GCD` `из` 5". После этого скрипт опять выводит в режим ожидания двух натуральных чисел. Если в какой-то момент работы пользоваться `exit` вместо этого лучше не стоит, так можно навредить на экран "`bye`" и закончить свою работу.

Выполнение `НОД` можно реализовать с помощью алгоритма Евклида. Если кратко написать функцию `gcd`, которая принимает на вход два аргумента (базиса `M` и `N`). Если аргументы равны, то мы нашли `НОД` — он равен `M` (или `N`), иначе выводить соответствующее сообщение на экран (или `exit`). Иначе нужно сравнить аргументы между собой. Если `M` больше `N`, то вызовем ту же функцию `gcd`, но в качестве первого аргумента передадим `(M-N)`, а в качестве второго `N`. Если же наоборот, то вызовем функцию `gcd` с первым аргументом `M`, а вторым `(M-N)`.

Пример корректной работы скрипта:

```
./gcdscript.sh
10 15
GCD is 5
7 5
GCD is 1
bye
```

Примечание: в вызове функции из себя самой нет ничего страшного или неправильного, т.е. можно вызывать `gcd` прямо внутри `gcd`!

Примечание 2: для завершения работы функции в произвольном месте можно использовать инструкцию `return` (как инструкции функции после `return` выполняться не будут). В отличие от `exit` эта команда завершит только функцию, а не выполнение всего скрипта целиком. Однако в редкой версии можно обойтись и без использования `return`.

Подсказка: в случае проблемы с решением задачи, обратитесь к [нашим рекомендациям по написанию скриптов](#).

Напишите программу. Тестируется через `stdin → stdout`

✓ Тестовый

Теперь вы можете загрузить [свои решения](#), где вы можете сравнить свое решение с другими или отправить на сайт.

```
1 while { true }
2 do
3   read x2 x3
4   if [ -z "$x2" ]; then
5     echo "bye"
6     break
7   fi
8   gcd () {
9     gcdmod=$((x2 % x3))
10    if [ $gcdmod -eq 0 ]; then
11      echo "bye"
12      return 0
13    fi
14    while { $gcdmod != 0 }
15    do
16      gcdmod=$((x2 % x3))
17    done
18    x2=$gcdmod
19    x3=$gcdmod
20  }
21  gcd $x2 $x3
22  echo "GCD is $x3"
23  return 0
24 done
25
26
27
28
```

Следующий шаг

Решить снова

Задание 19

Выполнение 20 задания (рис. 20).

Напишите калькулятор на Bash. При запуске ваш скрипт должен ожидать ввода пользователем команды (при этом на экран выводить ничего не нужно). Команды могут быть трех типов:

1. Слово "exit". В этом случае скрипт должен вывести на экран слово "bye" и завершить работу.
2. Три аргумента через пробел - первый операнд (целое число), оператор (одно из "+", "-", "*", "/", "%", "sqrt") и второй операнд (целое число). В этом случае нужно проанализировать операцию над заданными числами и вывести результат на экран. После этого перейти в режим ожидания новой команды.
3. Любая другая команда на одного аргумента или на три аргумента, но с оператором на первом. В этом случае нужно вывести на экран слово "error" и завершить работу.

Чтобы проверить работу скрипта, вы можете вызвать сразу несколько команд в файле и передать его скрипту на вход (т.е. выполнить `./calculator.sh < script.txt`). В этом случае он должен вывести сразу все ответы на экран.

Например, если входной файл будет следующего содержания:

```
10 + 5
2 * 10
exit
```

то на экране будет:

```
15
200
bye
```

Если же не ввод поступит следующий файл:

```
5 - 5
2/10
exit
```

то на экране будет:

```
0
error
```

т.е. вторая команда была некорректной (в ней всего один аргумент, т.е. нет пробела между числом и оператором), а единственная допустимая команда на одного аргумента это "exit".

Подсказка: в случае проблемы с решением задан, обратитесь внимание на наши рекомендации по написанию скриптов.

Напишите программу. Тестируется через stdin → stdout

✔ Правильно.

Посмотрите на доступные ресурсы, где вы можете проверить свое решение с другими или получить совет.

```
1 #!/bin/bash
2 while [[ $1 != "" ]]
3 do
4     read -a arr < /dev/null
5     if [[ $arr[0] == "exit" ]]
6     then
7         echo "bye"
8         break
9     elif [[ "${arr[0]}" =~ "[0-9]+-[0-9]" || "${arr[0]}" =~ "[0-9]+/[0-9]" ]]
10    then
11        echo "error"
12        break
13    else
14        case $arr[0] in
15            "+") let "result = ${arr[1]} + ${arr[2]}" ;;
16            "-") let "result = ${arr[1]} - ${arr[2]}" ;;
17            "*" ) let "result = ${arr[1]} * ${arr[2]}" ;;
18            "/" ) let "result = ${arr[1]} / ${arr[2]}" ;;
19            "%" ) let "result = ${arr[1]} % ${arr[2]}" ;;
20            "sqrt") let "result = $(echo "scale=2; sqrt(${arr[1]})" | bc)" ;;
21            *) echo "error" ; break ;;
22        esac
23        echo $result
24    fi
25 done
```

Следующий шаг

Решить снова

Задание 20

Выполнение 21 задания (рис. 21).

Пусть в директории `/home/bi` лежат файлы `Star_Wars.avi`, `star_trek OST.mp3`, `STARS.txt`, `stardust.mpeg`, `Eddard_Stark_biography.txt`.

Отметьте все файлы, которые **найдет** команда `find /home/bi -iname "star*"`, но **НЕ найдет** команда `find /home/bi -name "star*"` ?

Выберите все подходящие ответы из списка

✔ Хорошая работа.

- ☒ STARS.txt
- ☐ star_trek OST.mp3
- ☐ Eddard_Stark_biography.txt
- ☒ Star_Wars.avi
- ☐ stardust.mpeg

Следующий шаг

Решить снова

Задание 21

Выполнение 22 задания (рис. 22).

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте **все верные** утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

☒ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Если заменить в команде поиска `-name`, на `-path`, то результат поиска всегда останется неизменным
- ☐ Опция `-path` используется только для поиска директорий, а `-name` только для поиска файлов
- ☐ Опция `-path` аналогична `-name`, но игнорирует размер букв (строчные/прописные) в имени файла
- ☒ В некоторых случаях `find` с `-name` найдет меньше файлов, чем `find` с таким же запросом, но с `-path`
- ☒ Если заменить в команде поиска `-name`, на `-path`, то результат поиска иногда может остаться таким же

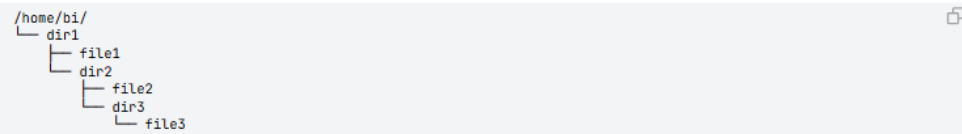
Следующий шаг

Решить снова

Задание 22

Выполнение 23 задания (рис. 23).

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:



Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*" ?`

Выберите один вариант из списка

☒ Верно.

- ☐ Только `file1`
- ☐ Только `file2`
- ☒ Все кроме `file3`
- ☐ Все кроме `file1`
- ☐ Все кроме `file2`

Следующий шаг

Решить снова

Задание 23

Выполнение 24 задания (рис. 24).

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем **в каждой строке есть слово "word"**. Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один вариант из списка

✓ Отличное решение!

- ☐ Все, кроме `grep "word" file.txt > results.txt`
- ☒ `results.txt` будет одинакового размера во всех случаях
- ☐ `grep -C 1 "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt` и `grep -B 1 "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt`

Следующий шаг

Решить снова

Задание 24

Выполнение 25 задания (рис. 25).

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xkLXKL]?[uU]buntu$" text.txt`.

Выберите все подходящие ответы из списка

✓ Правильно, молодец!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Mac OS X 10.9, Windows XP, Ubuntu 12.04
- ☒ The best OS is Xubuntu
- ☒ I prefer Kubuntu
- ☒ Hmm, XKLubuntu
- ☒ Lubuntu is better than Ubuntu
- ☒ Linux is not always Ubuntu

Следующий шаг

Решить снова

Задание 25

Выполнение 26 задания (рис. 26).

Что произойдет, если в команде `sed -n "[a-z]*p" text.txt` не указывать опцию `-n`?

Выберите один вариант из списка

✓ Отличное решение!

- ☐ Будут выведены все строки файла `text.txt`, в которых есть только большие буквы латинского алфавита
- ☐ На экран будет выведено всё содержимое файла `text.txt`
- ☒ Каждая строчка будет выведена два раза
- ☐ Появится сообщение об ошибке

Следующий шаг

Решить снова

Задание 26

Выполнение 27 задания (рис. 27).

Запишите в форму ниже инструкцию `sed`, которая заменит все "аббревиатуры" в файле `input.txt` на слово "abbreviation" и запишет результат в файл `edited.txt` (на экран при этом ничего выводить не нужно). Обратите внимание, что в инструкции **должны быть** указаны и сам `sed`, и оба файла!

Под "аббревиатурой" будем понимать слово, которое удовлетворяет следующим условиям:

- состоит только из больших букв латинского алфавита,
- состоит из хотя бы двух букв,
- окружено одним пробелом с каждой стороны.

При этом будем считать, что в тексте **не может быть две "аббревиатуры" подряд**. Например, текст `" YOU YOU and YOU!"` является **некорректным** (в нем есть две "аббревиатуры", но они идут подряд) и на таких примерах мы проверять вашу инструкцию **не будем**.

Пример: если у вас был текст `"Hi, I heard these songs by ABBA, TLA and DM !"`, то он должен быть преобразован в `"Hi, I heard these songs by ABBA, abbreviation and abbreviation !"`.

Примечание: после вашей замены "аббревиатуры" на слово "abbreviation" **количество пробелов** в тексте **не должно меняться!**

Внимание! Во время проверки **мы не запускаем команду**, которую вы ввели на реальном файле с "аббревиатурами" (это небезопасно, можно же ввести `rm -rf /*`!) Вместо этого мы сперва анализируем структуру вашей инструкции (например, что в ней использован именно `sed` и сделано это ровно один раз, что на вход подается `input.txt`, а результат будет записан в `edited.txt` и т.д.), а затем **запускаем её смысловую часть** (т.е. поиск по регулярному выражению и замена на "abbreviation") на тестовых примерах. К сожалению, наш запуск **не идеально повторяет** `sed`, но он очень близок к нему. Главная "несовместимость" заключается в том, что наша проверка не понимает идущие подряд символы, отвечающие за количество повторений (т.е. *, +, ? и {}). Однако эту "несовместимость" легко исправить указав при помощи "(" и ")" какой из символов к чему относится! Например, регулярное выражения `a+?` (ноль или один раз по одной или более букве "a") нужно записать как `(a+)?` (при этом запись `(a)+?`, конечно же, не поможет).

Напишите текст

✓ Прекрасный ответ.

```
sed -r 's/[A-Z]{2,} / abbreviation /g' input.txt > edited.txt
```

Следующий шаг

Решить снова

Задание 27

Выполнение 28 задания (рис. 28).

Вы можете скачать и попробовать применить `gnuplot` к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске `gnuplot`, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

✓ Правильно.

- ☒ `-p, -persist`
- ☐ Графики и так не закрываются автоматически при закрытии `gnuplot`!
- ☐ `-raise`
- ☐ Такой опции не существует

Следующий шаг

Решить снова

Задание 28

Выполнение 29 задания (рис. 29).

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете `gnuplot` и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет **название** у построенного **ряда данных** и **сколько** будет нарисовано **точек** на графике?

Выберите один вариант из списка

✓ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☐ Название – первое значение из второго столбца, нарисовано 10 точек
- ☐ Название "data.csv" using 1:2", нарисовано 10 точек
- ☐ Название "no name", нарисовано 10 точек
- ☐ Название – первое значение из первого столбца, нарисовано 10 точек

Следующий шаг

Решить снова

Задание 29

Выполнение 30 задания (рис. 30).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [plot.gnu](#), [plot_advanced.gnu](#), [plot_advanced2.gnu](#). Все три скрипта основаны на [этой заметке](#), данные также взяты оттуда.

Предположим, что вы пишете gnuplot-скрипт и у вас в нем есть три переменные `x1`, `x2`, `x3`, в которых записаны координаты важных точек по оси OX (по возрастанию). Вы хотите, чтобы на этой оси было только три деления (т.е. три черточки) в этих самых координатах, а подписи этих делений были оформлены в виде "point <номер точки>, value <значение соответствующей переменной>". Например, для `x1=0`, `x2=10`, `x3=20`, это были бы надписи "point 1, value 0" в точке с координатой 0 по горизонтали, "point 2, value 10" в точке с координатой 10 и "point 3, value 20" в точке с координатой 20. Или, например, `x1=100`, `x2=150`, `x3=250`, это были бы надписи "point 1, value 100" в точке с координатой 100, "point 2, value 150" в точке с координатой 150 и "point 3, value 250" в точке с координатой 250.

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно добавить в скрипт, для выполнения этой задачи.

Примечание: проверять, что переменные `x1`, `x2`, `x3` идут по возрастанию или что они являются числами **не нужно**!

Примечание 2: в видеофрагменте на предыдущем шаге звучал термин *конкатенация*, который важен для выполнения данного задания. Под конкатенацией обычно понимают "склеивание" двух строк в одну длинную строку, например, конкатенация строк "Данные из файла " и "data.csv" даст строку "Данные из файла data.csv".

Подсказка: настоятельно рекомендуем изучить примеры скриптов – в них есть большая часть решения!

Напишите текст

✓ Правильно, молодец!

```
set xtics ("point 1, value "x1 x1 , "point 2, value "x2 x2 , "point 3, value "x3 x3)
```

Следующий шаг

Решить снова

Задание 30

Выполнение 31 задания (рис. 31).

Если вы не скачали на предыдущем шаге файлы [animated.gnu](#) и [move.rot](#), то скачайте их теперь, т.к. они понадобятся для выполнения задания.

Указанные файлы использовались в последнем видеофрагменте для создания вращающегося графика. Измените инструкции в файле `move.rot` (т.е. **добавлять** и **удалять** инструкции **нельзя!**) таким образом, чтобы:

- График **отразился зеркально** относительно горизонтальной поверхности. То есть там, где была точка (10, 10, 200), станет точка (10, 10, -200), где была точка (-10, -10, 200) станет (-10, -10, -200) и т.д. При этом точка (0, 0, 0) останется на месте.
- Изображение стало **вращаться в обратную сторону**. То есть если раньше вращалось "влево", то теперь станет "вправо".
- Вращение стало **в два раза быстрее**. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Измененный файл загрузите в форму ниже.

Примечание: наша система проверки **не может** запустить на вашем файле `move.rot` программу gnuplot и сравнить полученный график с заданным. Вместо этого **мы анализируем команды**, которые вы указали в файле. Поэтому если вы видите, что ваш скрипт в gnuplot работает точно по условию, а мы отвечаем "Incorrect/Неверно", то попробуйте упростить свою модификацию `move.rot` и отправить его еще раз.

Напишите текст

✓ Верно.

```
a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.1
if (a<50) reread
```

Следующий шаг

Решить снова

Задание 31

Выполнение 32 задания (рис. 32).

Какая команда(ы) установят файлу `file.txt` права доступа `rwXrw-r--`, если изначально у него были права `r--r--r--`. Укажите **все** верные варианты ответа!

Примечание: запись вида команда1; команда2; команда3 означает, что в терминале последовательно выполнялись все три команды (сначала команда1, затем команда2 и, наконец, команда3).

Выберите все подходящие ответы из списка

☒ Абсолютно точно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ `chmod u-wx file.txt; chmod g-w file.txt`
- ☒ `chmod u+wx file.txt; chmod g+w file.txt`
- ☐ `chmod o-wx file.txt; chmod g-x file.txt; chmod a+wx file.txt`
- ☐ `chmod 777 file.txt`
- ☐ `chmod rwxrw-r-- file.txt`
- ☒ `chmod 764 file.txt`

Следующий шаг

Решить снова

Задание 32

Выполнение 33 задания (рис. 33).

Предположим вы использовали команду `sudo` для создания директории `dir`. По умолчанию для `dir` были выставлены права доступа `rwXr-x` (владелец root, группа root). Таким образом никто кроме пользователя root не может ничего записывать в эту директорию, например, не может создавать файлы в ней.

После выполнения какой команды user из группы group всё-таки сможет создать файл внутри `dir`? Укажите **все** верные варианты ответов!

Примечание: считаем, что все команды выполняются от имени user, если явно не указано, что команда выполнена с `sudo`.

Примечание 2: мы выбрали пример с директорией, а не с файлом не случайно.

Дело в том, что если создать при помощи `sudo` файл с правами `rw-r--r--` в директории, которая принадлежит пользователю, то возникнет любопытная ситуация. С одной стороны пользователь может удалить этот файл (т.к. ему разрешено удалять все файлы внутри его директории) и может прочитать его содержимое (т.к. право `r` у файла установлено для всех), с другой стороны он не может этот файл редактировать (т.к. право `w` у файла есть только для root). При этом некоторые "умные" редакторы, например, `vim` позволяют даже редактировать этот файл, но делают они это своеобразно: через удаление оригинала и создание копии уже с нужными правами (удалять мы можем, а раз можем читать, то и копию создать не сложно). Итого получается, что несмотря на права `rw-r--r--`, пользователь может сделать с этим файлом почти всё что угодно!

В случае же, когда речь идет о директории созданной root, ситуация будет проще: пользователь сможет смотреть её содержимое (у него есть право `r`), но удалять и создавать файлы в ней не сможет (права `w` у него нет).

Важно отметить, что директории в Linux это в каком-то смысле файлы. Содержимое такого "файла" — это записи о файлах и поддиректориях этой директории (грубо говоря их названия). Таким образом, право `r` у директории дает возможность просматривать "записи", т.е. просматривать её состав. Право `w` у директории дает возможность удалять/добавлять новые "записи", т.е. удалять/создавать файлы/поддиректории в ней.

На самом деле и это еще не всё. Существует так называемый [sticky bit](#) (атрибут файла или директории), выставление которого меняет описанное выше поведение. Файлы (или директории) с таким атрибутом сможет удалить только их владелец вне зависимости от прав, установленных у директории, в которой эти файлы (или директории) лежат!

Отдельное спасибо слушателю курса [Alexey Antipovskiy](#) за помощь в оформлении Примечания 2!

Выберите все подходящие ответы из списка

☒ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `sudo chown user dir`
- ☒ `sudo chmod a+w dir`
- ☒ `sudo chown user:group dir`
- ☐ `sudo chmod g+w dir`
- ☐ `sudo chmod o+x dir`
- ☒ `sudo chmod a+w dir`

Следующий шаг

Решить снова

Задание 33

Выполнение 34 задания (рис. 34).

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Выберите все подходящие ответы из списка

☒ Отлично!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Количество символов
- ☒ Количество слов
- ☒ Количество строк
- ☒ Длину самой длинной строки
- ☒ Размер файла в байтах

Следующий шаг

Решить снова

Задание 34

Выполнение 35 задания (рис. 35).

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести в **удобном для чтения формате** (например, вместо 2948 байт надо выводить 2.9К) и **больше** на экран выводить **ничего не нужно**). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по 800 Кбайт и две поддиректории в каждой из которой лежит по файлу в 400 Кбайт, то загаданная команда должна вывести на экран одно число: 2.4М (также на экране может быть выведен еще и символ ".", обозначающий, что это размер именно текущей директории).

Напишите текст

☒ Здорово, всё верно.

`du -h -s`

Следующий шаг

Решить снова

Задание 35

Выводы

Успешно выполнена 1 часть курса на stepic